파이썬 완전 백과사전 2권 · 딥러닝 심화: 언어와 생성 교안

1일차

순서가 있는 데이터: RNN과 LSTM

1~5차시 · 책 11장 RNN과 LSTM

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/11-rnn-lstm

과정 전체: 25차시, 5일

일	차시	주제	책 범위
1일	1~5	순서가 있는 데이터: RNN과 LSTM	11장
2일	6~10	어텐션과 트랜스포머, 작은 GPT	12장
3일	11~15	한국어 자연어 처리: 토큰화부터 미세조정	13장
4일	16~20	사전학습 언어 모델과 LoRA 미세조정	14장
5일	21~25	생성 모델: 오토인코더, VAE, GAN, 확산	15장

선수 과정 = 딥러닝 기초(4~10장). 텐서, autograd, nn.Module, DataLoader, 학습 요령, CNN, 전이 학습

흐름 = 순서를 읽는 모델 → 한꺼번에 돌아보는 모델 → 언어 → 거대 언어 모델 → 만들어 내는 모델

한 차시 50분 · 하루 5차시 · 교재: 파이썬 완전 백과사전 2권 11~15장 · 예제: examples/vol2

오늘의 시간표

차시	주제	책 범위
1	순서 데이터의 모양, RNN 셀	11장 들어가며 ~ 11.2
2	괄호 짝 맞추기, 기울기 소실과 폭주	11.3 ~ 11.4
3	LSTM, GRU와 파라미터 비용, 긴 기억 시험	11.5 ~ 11.7
4	층 쌓기·양방향, 꾸러미, 이산화탄소 예측	11.8 ~ 11.10
5	속담 짓는 글자 LSTM 실습, 11장 정리	11.11 ~ 11.12

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2 폴더에서 uv sync · 11장 예제 27개 모두 합쳐 약 90초 · 내려받을 데이터 없음

1차시

01

순서 데이터의 모양과 RNN 셀

책 11장 들어가며 ~ 11.2절

학습 목표

- 순서만 다른 데이터를 개수로는 구별할 수 없음을 확인한다
- 창을 밀며 잘라 (배치, 길이, 특성) 모양을 만들고 읽는다
- RNN 셀을 손으로 짜고 nn.RNN과 같은 값이 나오는지 맞춘다

11장 순서를 바꾸면 뜻이 바뀐다

```
리뷰1 = "가격은 별로지만 맛은 최고"
리뷰2 = "맛은 별로지만 가격은 최고"
# 글자마다 몇 번 나왔는지만 센다
print(Counter(리뷰1) == Counter(리뷰2))
print(torch.equal(개수_벡터(리뷰1),
                  개수_벡터(리뷰2)))

오르는_중 = torch.tensor([100.0, 102.0,
          105.0, 109.0, 114.0])
내리는_중 = 오르는_중.flip(0)  # 거꾸로
```

```
True
True
106.0 106.0
True
마지막 날 - 첫날:
  14.0 -14.0
```

두 리뷰는 글자 종류와 개수가 **완전히 같다**. 오르는 주가와 내리는 주가는 평균도 표준편차도 같다. 다른 것은 **순서**뿐

11.1 창을 밀며 잘라 내기

```
기온 = torch.tensor([3.0, 4.5, 6.0, 5.5, 7.0,
        8.5, 8.0, 9.5, 11.0, 10.5])
창 = 3
조각들 = 기온.unfold(0, 창, 1)
print(조각들.shape)
print(조각들[:3])

X = 조각들[:-1]  # 마지막 창은 정답이 없다
y = 기온[창:]    # 각 창의 바로 다음 날

X = X.unsqueeze(-1)  # 하루에 숫자 1개
print("입력:", tuple(X.shape),
      "정답:", tuple(y.shape))
```

```
torch.Size([8, 3])
tensor([[3.0000, 4.5000, 6.0000],
        [4.5000, 6.0000, 5.5000],
        [6.0000, 5.5000, 7.0000]])
[3.0, 4.5, 6.0] → 5.5
[4.5, 6.0, 5.5] → 7.0
[6.0, 5.5, 7.0] → 8.5
입력: (7, 3, 1) 정답: (7,)
같은 메모리를 쓰나? True
```

사흘짜리 **창**을 하루씩 밀면 문제와 정답 쌍이 생긴다. unfold도 view처럼 복사 없이 같은 메모리를 다르게 읽는다

11.1 (배치, 길이, 특성)으로 읽기

기온만

`(7, 3, 1)`

문제 7개, 각각 사흘, 하루마다 숫자 1개

기온·습도·바람

`(7, 3, 3)`

하루에 보는 숫자가 셋이면 특성이 3

문장

`(문장 수, 글자 수, …)`

마지막 자리는 글자 하나를 나타내는 숫자의 개수

순환 신경망의 입력은 언제나 **3차원**. 자리마다 이름을 붙여 읽는 버릇이 오늘 내내 쓰인다

11.2 메모장을 고쳐 쓰며 읽기

"지금 읽은 것과 지난 메모를 섞어서 **새 메모**를 쓴다"

```
새 메모 h_t = tanh( x_t @ W_x  +  h_(t-1) @ W_h  +  b )
                    지금 읽은 것      지난 메모
```

- 회의록 쓰는 사람: 발언 하나를 들을 때마다 **요약을 다시 쓴다**
- 요약 칸의 크기는 정해져 있다: 중요한 것은 남기고 덜 중요한 것은 밀어낸다
- 이 메모가 **은닉 상태(hidden state)** h. tanh가 -1~1 사이로 묶어 준다

11.2 손으로 만드는 RNN 셀

```
W_x = torch.randn(입력_크기, 메모_크기) * 0.5   # 3 → 4
W_h = torch.randn(메모_크기, 메모_크기) * 0.5   # 4 → 4
def rnn_셀(x, h):
    return torch.tanh(x @ W_x + h @ W_h + b)
순서열 = torch.randn(2, 5, 입력_크기)   # (배치 2, 길이 5, 특성 3)
h = torch.zeros(2, 메모_크기)           # 처음에는 빈 메모장
for t in range(순서열.shape[1]):       # 한 시점씩 차례로
    h = rnn_셀(순서열[:, t], h)
```

```
t=0: 첫 문장의 메모 [0.034, 0.412, -0.082, -0.311]
t=1: 첫 문장의 메모 [0.419, 0.388, -0.072, 0.271]
t=2: 첫 문장의 메모 [0.203, -0.596, -0.832, 0.29]
t=3: 첫 문장의 메모 [0.395, -0.032, -0.8, -0.651]
t=4: 첫 문장의 메모 [-0.498, -0.029, 0.382, 0.437]
모든 시점의 메모: (2, 5, 4) 마지막 메모: (2, 4)
```

11.2 nn.RNN: 같은 일을 하는 부품

```
rnn = nn.RNN(input_size=3, hidden_size=4,
             batch_first=True)
# 같은 가중치로 손 계산 (x @ W.T 꼴)
for t in range(5):
    h = torch.tanh(
      순서열[:, t] @ rnn.weight_ih_l0.T
      + rnn.bias_ih_l0
      + h @ rnn.weight_hh_l0.T
      + rnn.bias_hh_l0)

출력, h_n = rnn(순서열)
```

```
weight_ih_l0   (4, 3)
weight_hh_l0   (4, 4)
bias_ih_l0     (4,)
bias_hh_l0     (4,)
출력: (2, 5, 4) h_n: (1, 2, 4)
손 계산과 차이: 0.0
출력의 마지막 시점 == h_n?
  True
```

ih = 입력 → 메모, **hh** = 메모 → 메모, **l0** = 0번 층. 손 계산과 차이 **0.0**: 같은 계산이다

11.2 nn.RNN이 돌려주는 두 가지

출력 (2, 5, 4)

모든 시점의 메모

시점마다 무언가를 내야 하는 문제
예: 다음 글자 맞히기 (5차시)

h_n (1, 2, 4)

마지막 시점의 메모

다 읽고 한 번 판단하는 문제
예: 괄호 짝 판정 (2차시)

h_n 맨 앞의 1은 **층 수**. 층이 하나면 `출력[:, -1]`과 `h_n[0]`은 같은 값

11.2 길이가 달라도 같은 모델

```
for 길이 in [1, 5, 500]:
    출력, h_n = rnn(torch.randn(2, 길이, 3))

# 처음 메모장: (층 수, 배치, 메모 크기)
h0 = torch.zeros(1, 2, 4)
앞부분, h_중간 = rnn(torch.randn(2, 3, 3), h0)
뒷부분, h_끝 = rnn(torch.randn(2, 2, 3),
                   h_중간)  # 이어서 읽기

# batch_first를 빼면 (길이, 배치, 특성)
rnn_기본 = nn.RNN(input_size=3, hidden_size=4)
출력, _ = rnn_기본(torch.randn(5, 2, 3))
```

```
파라미터 수: 36
길이   1: 출력 (2, 1, 4)
길이   5: 출력 (2, 5, 4)
길이 500: 출력 (2, 500, 4)
h_n은 셋 다 (1, 2, 4)
이어 읽은 출력:
  (2, 3, 4) (2, 2, 4)
batch_first=False 출력:
  (5, 2, 4)
```

파라미터 36개 = W_x 12 + W_h 16 + 편향 8. 이 책은 언제나 **batch_first=True**

직접 실행 10분

1. b01_order.py와 b04_nn_rnn.py를 실행하고, 손 계산과 nn.RNN의 차이가 0.0인지 확인한다
2. b02_window.py 복사본에서 창 = 3을 창 = 5로 바꾼다. 조각 수와 입력 모양은?
3. b05_length.py 복사본에서 rnn의 hidden_size=4를 8로(h0도 (1, 2, 8)로) 바꾼다. 파라미터 수는? 먼저 손으로 계산해 본다

```
cd examples/vol2/11-rnn-lstm && uv run python b04_nn_rnn.py
```

nn.RNN(3, 4, batch_first=True)에 길이 500짜리 입력 2개를 넣으면 h_n의 모양은?

Q

가 (2, 500, 4)

나 (1, 2, 4)

다 (500, 2, 4)

라 길이가 너무 길어 오류

1차시 정리

순서만 다른 데이터는 개수로 구별할 수 없다. 순서 데이터는 **(배치, 길이, 특성)**, 창을 밀며 잘라 만든다

RNN: **h_t = tanh(x_t @ W_x + h_(t-1) @ W_h + b)**. 모든 시점이 같은 가중치라 어떤 길이든 받는다

출력은 모든 시점의 메모, **h_n**은 마지막 메모. batch_first=True를 잊지 않는다

다음 차시: 순서를 알아야만 풀리는 문제, 그리고 RNN의 약점

2차시

02

괄호 짝 맞추기, 기울기 소실과 폭주

책 11.3 ~ 11.4절

학습 목표

- 순서가 필요한 문제에서 MLP와 RNN의 차이를 숫자로 본다
- 길이를 받아 주는 것과 처음 보는 길이에서 잘하는 것을 가른다
- 시점별 기울기를 재어 소실과 폭주를 확인하고 처방을 고른다

11.3 순서를 알아야 풀리는 문제

```
def 짝맞나(괄호: str) -> bool:
    깊이 = 0
    for 글자 in 괄호:
        깊이 += 1 if 글자 == "(" else -1
        if 깊이 < 0:
            return False
    return 깊이 == 0
```

`(()())` 짝 맞음

`())(()` 짝 틀림

둘 다 여는 괄호 셋, 닫는 괄호 셋

앞에서부터 읽으며 **열린 괄호의 깊이**를 센다. 0 아래로 내려가는 순간이 있으면 틀린 것

11.3 개수로는 절대 못 가르는 데이터

```
괄호들, 정답 = 괄호_만들기(3000,
        길이들=[20], 씨앗=0)
for 괄호, 답 in zip(괄호들[:4],
                  정답[:4].tolist()):
    print(괄호, "짝 맞음" if 답
          else "짝 틀림")

X, 길이 = 원핫으로(괄호들[:2])
print("원-핫 모양:", tuple(X.shape))
print(X[0, :3])
```

```
((()(((())(()))))()) 짝 맞음
()()()()()())((()()) 짝 틀림
()(((()))(())()(())) 짝 맞음
()())(()(()())))(()() 짝 틀림
짝 맞음 비율: 0.5
'('와 ')' 개수가 늘 같은가: True
원-핫 모양: (2, 20, 3)
tensor([[0., 1., 0.],
        [0., 1., 0.],
        [0., 1., 0.]])
```

틀린 괄호는 짝 맞는 괄호에서 두 글자를 맞바꿔 만들었다: **여는 괄호 10개, 닫는 괄호 10개**로 언제나 같다

11.3 세 모델의 대결

개수만 보는 MLP

글자를 모두 더해 여는 괄호·닫는 괄호 개수만 본다

검증 50.0%

파라미터 194개 · 학습 50.0%

펼쳐서 보는 MLP

20글자 × 3칸을 한 줄 60칸으로 펴서 넣는다

검증 99.7%

파라미터 4,034개 · 학습 100.0%

RNN

한 글자씩 읽으며 메모를 고쳐 쓰고 마지막 메모로 판단

검증 100.0%

파라미터 1,250개 · 학습 100.0%

순서를 버리면 원리적으로 풀 수 없다. 같은 학습: Adam lr=3e-3, 배치 64, 30에폭

11.3 자리마다 따로 배우기, 한 벌로 돌려 쓰기

펼친 MLP · 4,034개

`nn.Linear(20 * 3, 64)`

1번 자리, 2번 자리, ..., 20번 자리를 위한 가중치를 **따로따로** 배운다. 첫 층은 60칸 입력만 받는다

RNN · 1,250개

`nn.RNN(3, 32)`

모든 자리에 **같은 가중치**를 돌려 쓴다. 한 글자를 읽는 법 하나만 배우면 된다

질문: 학습 때 길이 20만 봤다. 길이 24, 30, 40짜리 괄호를 주면?

11.3 처음 보는 길이

```
for 길이 in [20, 24, 30, 40]:
    괄호들, 정답 = 괄호_만들기(1000, [길이], 씨앗=2)
    X, _ = 원핫으로(괄호들)
    정확도 = (모델(X).argmax(dim=1) == 정답).float().mean().item()
    펼친_MLP(X)    # try 안에서 실행해 오류를 잡는다
```

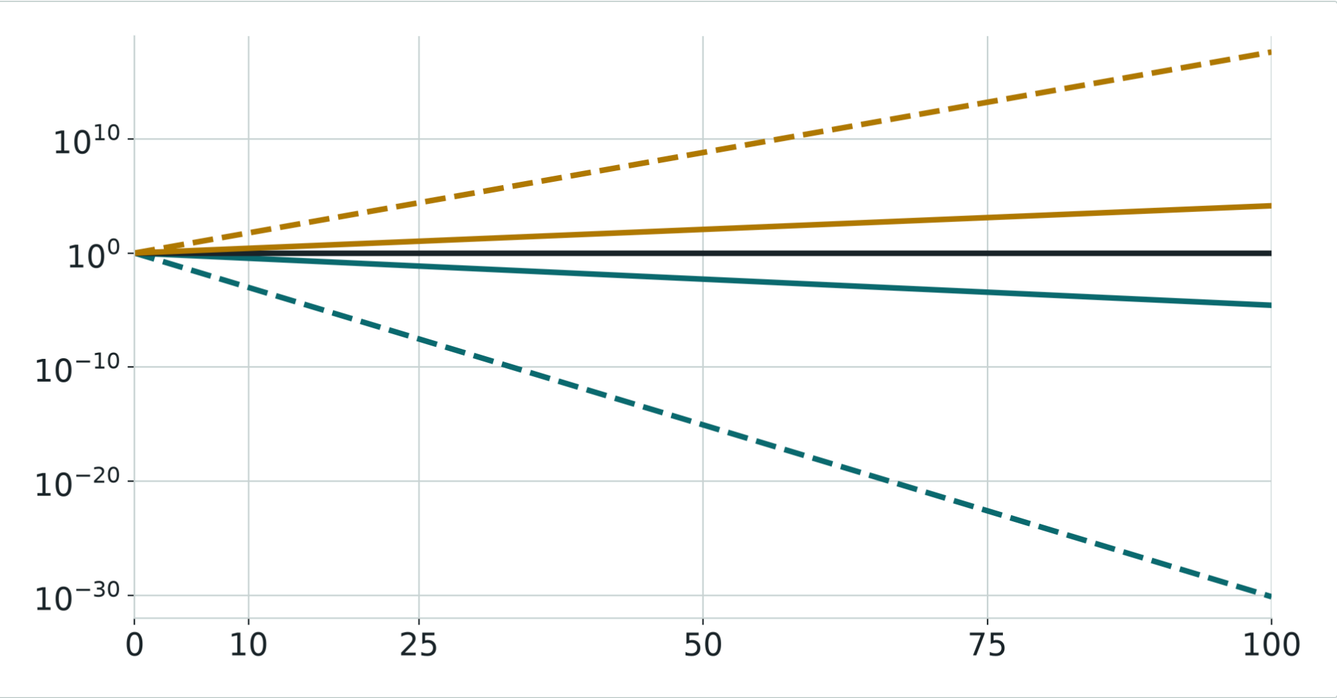
```
길이 20: RNN 정확도 100.0% / 펼친 MLP 실행됨
길이 24: RNN 정확도 86.6% / 펼친 MLP 모양 오류 (1000x72 and 60x64)
길이 30: RNN 정확도 57.5% / 펼친 MLP 모양 오류 (1000x90 and 60x64)
길이 40: RNN 정확도 48.2% / 펼친 MLP 모양 오류 (1000x120 and 60x64)
```

펼친 MLP는 길이가 바뀌면 **실행조차 안 된다**

RNN은 받아 주지만 길이 40에서 48.2%: 받는 것과 잘하는 것은 다르다

2차시 · 11.3 처음 보는 길이 · 예제 11-rnn-lstm/b08_paren_length.py (핵심 부분, 2초)

11.4 귓속말 놀이: 거듭 곱하기의 함정



- - 0.5 · — 0.9 · — 1.0 · — 1.1 · - - 1.5 · 가로는 곱한 횟수

0.9를 100번

2.66e-05

1.1을 100번

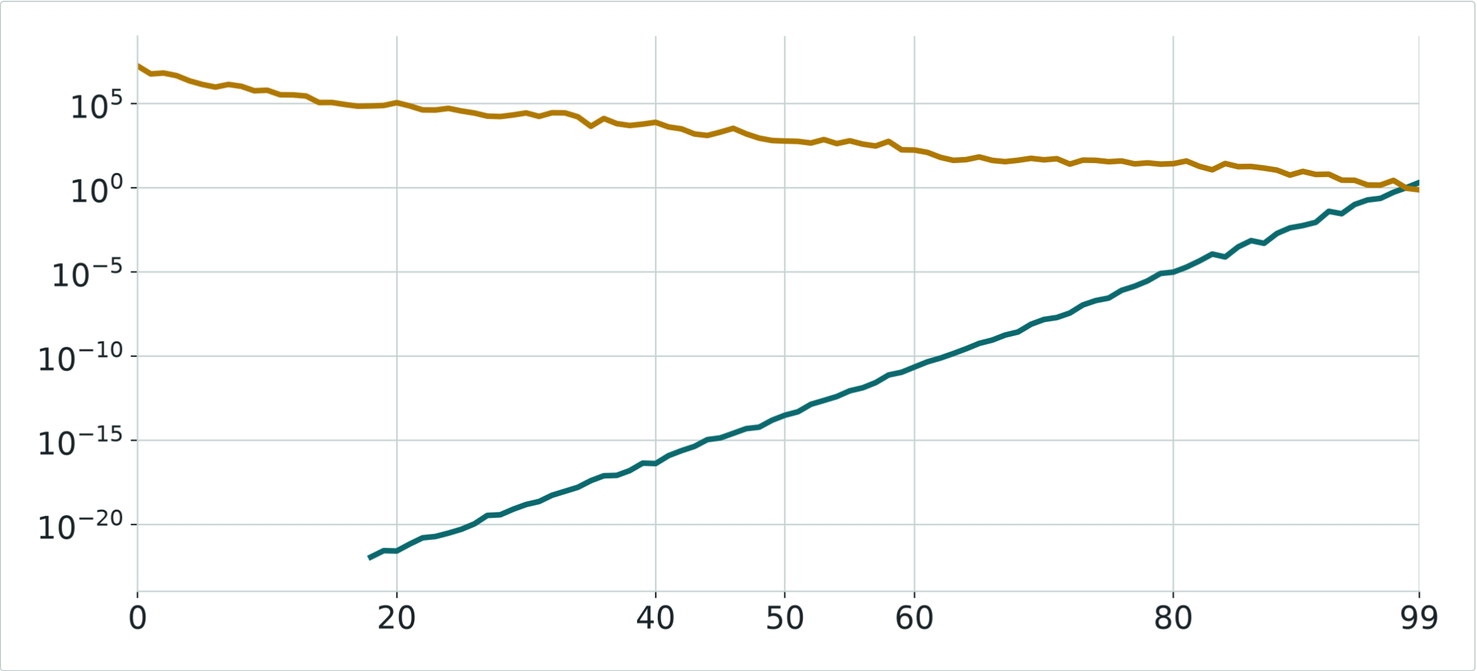
1.38e+04

tanh의 기울기 z=0, 1, 2, 3

1.0 · 0.42 · 0.0707 · 0.0099

2차시 · 11.4 기울기 소실과 폭주 · 책의 b09_power.py와 같은 계산을 선으로 그린 그림

11.4 시점별 기울기를 재 보기



— 기본 RNN · — W_h 4배 · 가로는 시점(99가 마지막)

기본 RNN, 시점 99 → 50

2.02 → 3.01e-14

기본 RNN, 시점 0

0.00e+00

W_h 4배, 시점 0

1.67e+07

기울기 소실: 층 대신 시간 방향으로. **기울기 폭주**: 같은 거듭 곱하기의 반대쪽

2차시 · 11.4 시점별 기울기 · 책의 b10_grad_time.py와 같은 모델·입력으로 100개 시점을 모두 그린 그림

11.4 두 문제의 처방은 다르다

폭주 → 클리핑

```
nn.utils.clip_grad_norm_(
    모델.parameters(), max_norm=1.0)
```

한 걸음의 크기에 상한을 둔다(8.9절). 순환 신경망 학습에서는 거의 기본으로 켠다

소실 → 구조를 바꾼다

작아진 기울기를 키울 방법은 없다. 클리핑으로는 해결되지 않는다

답은 다음 차시의 LSTM과 GRU

Q. tanh 대신 ReLU면? `nonlinearity="relu"`로 바꿀 수 있지만, 상한이 없어 메모가 커지고 폭주가 쉬워진다. W_h를 거듭 곱하는 구조는 그대로

직접 실행 10분

1. b07_paren_models.py와 b08_paren_length.py를 실행하고, 펼친 MLP의 모양 오류 메시지를 읽어 본다
2. b10_grad_time.py 복사본에서 기본 RNN을 nn.RNN(8, 32, batch_first=True, nonlinearity="relu")로 바꾼다. 시점 50의 기울기는 tanh보다 큰가?
3. b09_power.py 복사본의 배율 목록에 0.99와 1.01을 더한다. 100번 곱하면?

```
cd examples/vol2/11-rnn-lstm && uv run python b07_paren_models.py
```

2차시 · 직접 실행 · b07은 4초, b08은 2초, 나머지 1초 안팎 · 괄호 예제는 괄호도구.py와 같은 폴더에서

RNN 학습 중 손실이 갑자기 nan이 됐다. 먼저 해 볼 일은?

Q

가 메모 크기를 두 배로 키운다

나 clip_grad_norm_으로 걸음에 상한을 둔다

다 batch_first=False로 바꾼다

라 순서열을 더 길게 자른다

2차시 정리

괄호 짝 문제에서 개수만 보는 모델은 **50%**. 펼친 MLP는 길이가 바뀌면 쓸 수 없고, RNN은 한 벌의 가중치로 어떤 길이든 받는다

받아 주는 것과 **처음 보는 길이에서 잘하는 것**은 다르다(길이 40에서 48.2%)

시점마다 기울기를 곱하니 **사라지거나 터진다**. 폭주는 클리핑으로, 소실은 구조로

다음 차시: 기억을 따로 나르는 벨트, LSTM

3차시

03

LSTM, GRU와 긴 기억 시험

책 11.5 ~ 11.7절

학습 목표

- LSTM의 벨트와 문 셋을 손으로 계산하고 nn.LSTM과 맞춘다
- 덧셈으로 고쳐 쓰는 벨트가 기울기를 살리는 이유를 말한다
- RNN·GRU·LSTM의 비용과 긴 기억 성적을 숫자로 비교한다

11.5 LSTM: 기억을 따로 나르는 벨트

메모를 매번 새로 쓰지 말고, **지울 것만 지우고 더할 것만 더하자**

덜어 내기

벨트에서 필요 없는 물건을 뺀다

올려놓기

새 물건을 벨트에 싣는다

꺼내기

지금 필요한 물건만 꺼내 쓴다

```
c_t = f * c_(t-1) + i * g    # 벨트: 남길 만큼 남기고, 올릴 만큼 더한다
h_t = o * tanh(c_t)          # 메모: 꺼내 보일 만큼만
```

11.5 문 셋과 후보 하나

이름	하는 일	값의 범위
망각문 f	벨트 위의 기억을 얼마나 **남길지**	0~1 (시그모이드)
입력문 i	새 후보를 얼마나 **올려놓을지**	0~1 (시그모이드)
후보 g	올려놓을 새 내용	-1~1 (tanh)
출력문 o	벨트에서 지금 얼마나 **꺼내 보일지**	0~1 (시그모이드)

문은 0이면 닫힘, 1이면 활짝 열림. 0~1 사이 값을 만드는 함수가 시그모이드라 문마다 시그모이드를 쓴다

11.5 손으로 만드는 LSTM 셀

```
def lstm_셀(x, h, c):
    점수 = (x @ lstm.weight_ih_l0.T + lstm.bias_ih_l0
            + h @ lstm.weight_hh_l0.T + lstm.bias_hh_l0)   # (배치, 4 × 4)
    i, f, g, o = 점수.chunk(4, dim=1)  # PyTorch의 순서: 입력·망각·후보·출력
    입력문, 망각문, 출력문 = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    후보 = torch.tanh(g)
    c = 망각문 * c + 입력문 * 후보
    h = 출력문 * torch.tanh(c)
    return h, c
```

```
weight_ih_l0: (16, 3) weight_hh_l0: (16, 4)
출력: (2, 5, 4) h_n: (1, 2, 4) c_n: (1, 2, 4)
h 차이: 0.0  c 차이: 0.0
기억 c : [0.123, 0.341, -0.288, 0.479]
메모 h : [0.076, 0.158, -0.147, 0.244]
```

3차시 · 11.5 손으로 만드는 LSTM 셀 · 예제 11-rnn-lstm/b11_lstm_cell.py (핵심 부분)

11.5 왜 기울기가 살아남을까

RNN: 새로 만든다

`h_t = tanh(… h_(t-1) @ W_h …)`

거꾸로 갈 때마다 **W_h와 tanh의 기울기**가 곱해진다

LSTM: 고쳐 쓴다

`c_t = f * c_(t-1) + i * g`

거꾸로 갈 때 한 단계마다 곱해지는 것은 사실상 **f 하나**

"이건 기억해야 해"라고 판단하면 망각문을 1 가까이 연다. 그 값을 **모델이 배운다**

11.6 문을 두 개로 줄인 GRU

2014년. 벨트와 메모를 **하나로 합치고**, 문을 둘만 둔다

갱신문 z

지난 메모를 얼마나 남기고 새 후보로 얼마나 바꿀지. 망각문과 입력문을 하나로 묶었다

리셋문 r

새 후보를 만들 때 지난 메모를 얼마나 참고할지

```
새 메모 = (1 - z) * 새 후보 + z * 지난 메모
```

"남기는 양"과 "바꾸는 양"을 합쳐 1: 뭔가를 남기면 그만큼 덜 바꾼다

11.6 파라미터 비용: 한 벌, 세 벌, 네 벌

```
입력_크기, 메모_크기 = 8, 32
for 종류 in [nn.RNN, nn.GRU, nn.LSTM]:
    층 = 종류(입력_크기, 메모_크기, batch_first=True)
    출력, 상태 = 층(torch.randn(2, 10, 입력_크기))
```

종류	파라미터	가중치 줄 수	상태
RNN	1,344개	32	(1, 2, 32)
GRU	4,032개 (3벌)	96	(1, 2, 32)
LSTM	5,376개 (4벌)	128	[(1, 2, 32), (1, 2, 32)]

GRU는 문 둘 + 후보, LSTM은 문 셋 + 후보: RNN 한 벌의 계산을 **세 번, 네 번**

LSTM만 상태가 둘: `_, (h_n, c_n) = 모델(x)`

3차시 · 11.6 GRU와 파라미터 비용 · 예제 11-rnn-lstm/b12_params.py (핵심 부분, 출력을 표로)

11.7 긴 기억 시험: 멀리 떨어진 두 수 더하기

```
값 = torch.rand(개수, 길이, generator=g)
표시 = torch.zeros(개수, 길이)
앞 = torch.randint(0, 길이 // 2, …)
뒤 = torch.randint(길이 // 2, 길이, …)
표시[torch.arange(개수), 앞] = 1.0
표시[torch.arange(개수), 뒤] = 1.0
x = torch.stack([값, 표시], dim=-1)
y = (값 * 표시).sum(dim=1, keepdim=True)
```

```
숫자: [0.5, 0.77, 0.09, 0.13,
  0.31, 0.63, 0.49, 0.9,
  0.46, 0.63]
표시: [0, 1, 0, 0, 0, 0, 1, 0, 0, 0]
정답: 1.2583
늘 1.0이라고 답할 때 MSE: 0.1663
```

앞 절반의 표시된 숫자를 **줄 끝까지 기억**해야 풀린다. 입력 (개수, 길이, 2)

기준선 0.166: 이 근처면 아무것도 못 배운 것

3차시 · 11.7 더하기 문제 · 예제 11-rnn-lstm/b13_adding_data.py (핵심 부분)

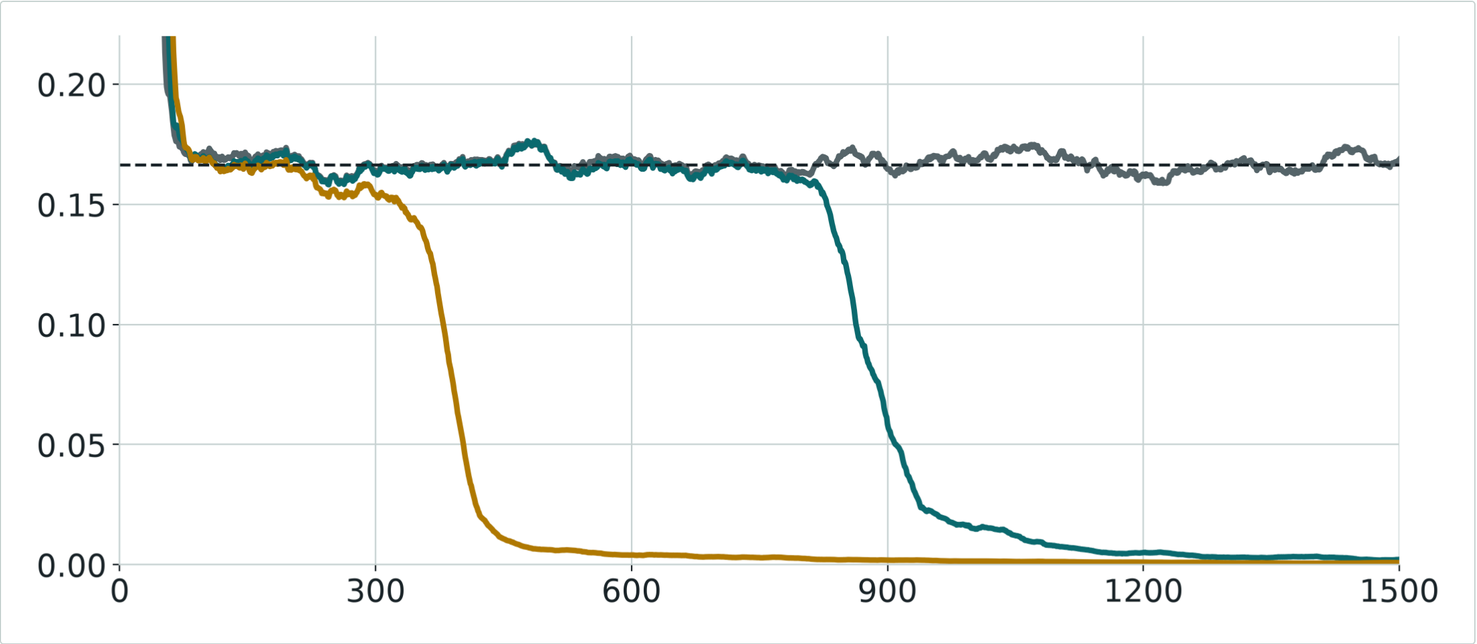
11.7 RNN, LSTM, GRU 비교

매번 새 문제 64개로 1,500번 학습(클리핑 1.0), 처음 보는 시험 문제 1,000개로 MSE

길이	모델	시험 MSE	첫 시점까지 닿는 기울기
10	RNN	0.0107	3.0e-01
10	LSTM	0.0002	3.5e-01
10	GRU	0.0002	3.4e-01
50	RNN	**0.1582** (기준선 0.166)	**2.5e-08**
50	LSTM	0.0026	2.7e-01
50	GRU	0.0005	8.7e-02

길이 50의 RNN은 **아무것도 배우지 못했다**. 학습을 마친 뒤에도 첫 시점의 기울기가 사실상 0

11.7 평평하다가 갑자기 내려가는 곡선



— RNN · — LSTM · — GRU · - - 기준선 0.166 · 가로는 학습 단계

LSTM 300·600·900번째
0.16 · 0.154 · 0.025

GRU 300·600번째
0.15 · 0.004

RNN 1,500번 내내
0.16 ~ 0.18

망각문을 여는 법을 배우는 데 시간이 걸린다. 초반에 손실이 안 줄어든다고 **너무 일찍 포기하지 않는다**

3차시 · 11.7 RNN, LSTM, GRU 비교 · 책의 b14_adding_train.py와 같은 설정(길이 50)으로 단계마다 손실을 기록해 그린 그림

직접 실행 10분

1. b14_adding_train.py 복사본에서 길이 목록을 [100]으로 바꿔 **먼저 실행을 걸어 둔다**(40초 안팎). 길이 100에서 셋의 시험 MSE는?
2. 기다리는 동안 b11_lstm_cell.py 복사본에서 i, f, g, o를 f, i, g, o로 바꾼다. h 차이와 c 차이는?
3. b12_params.py 복사본에서 메모_크기를 64로 바꾼다. 파라미터 수는 몇 배가 되나?

```
cd examples/vol2/11-rnn-lstm && uv run python 내실습_더하기.py
```

LSTM이 먼 과거까지 기울기를 전하는 핵심은?

가 파라미터가 RNN의 4배라서

나 벨트를 곱하고 **더해서** 고쳐 쓰고, 망각문을 배워서

다 tanh 대신 시그모이드를 써서

라 클리핑이 기본으로 들어 있어서

3차시 정리

LSTM: 벨트 c와 문 셋(망각·입력·출력). 덧셈으로 고쳐 쓰는 벨트 덕분에 기울기가 먼 과거까지 흐른다. 상태는 (h_n, c_n)

GRU: 문 둘(갱신·리셋), 파라미터는 RNN의 3배(LSTM은 4배). 무엇이 나은지는 검증 데이터로

길이 50 더하기: RNN **0.1582**(못 배움), LSTM 0.0026, GRU 0.0005. 곡선은 평평하다가 갑자기 내려간다

다음 차시: 층 쌓기·양방향, 길이가 다른 순서열, 실제 시계열 예측

3차시 · 정리 · 복습: 책 11.5 ~ 11.7절

4차시

04

층 쌓기, 꾸러미, 이산화탄소 예측

책 11.8 ~ 11.10절

학습 목표

- 층 수·방향 수에 따른 출력과 h_n의 모양을 읽는다
- pack_padded_sequence로 채운 칸을 건너뛰게 한다
- 기준선부터 세우고 시계열을 예측·이어 예측한다

11.8 두 층, 그리고 거꾸로 읽기

```
x = torch.randn(2, 7, 8)   # (배치 2, 길이 7, 특성 8)
설정들 = {"한 층": {}, "두 층": dict(num_layers=2, dropout=0.2),
          "양방향": dict(bidirectional=True), "두 층 양방향": dict(num_layers=2, bidirectional=True)}
lstm = nn.LSTM(8, 16, batch_first=True, **설정)
```

설정	출력	h_n	파라미터
한 층	(2, 7, 16)	(1, 2, 16)	1,664개
두 층	(2, 7, 16)	(**2**, 2, 16)	3,840개
양방향	(2, 7, **32**)	(**2**, 2, 16)	3,328개
두 층 양방향	(2, 7, **32**)	(**4**, 2, 16)	9,728개

h_n의 첫 자리 = **층 수 × 방향 수**. 양방향 출력은 정방향 16칸 + 역방향 16칸을 이어 붙인 것

11.8 양방향의 함정과 쓰면 안 되는 곳

```
출력, (h_n, _) = 양방향(x)
torch.allclose(h_n[0], 출력[:, -1, :16])
torch.allclose(h_n[1], 출력[:, 0, 16:])
# 분류에 쓸 때는 둘을 이어 붙인다
양쪽_요약 = torch.cat([h_n[-2], h_n[-1]],
                      dim=1)
```

```
정방향 마지막 = 출력[:, -1, :16]?
  True
역방향 마지막 = 출력[:, 0, 16:]?
  True
양쪽 요약: (2, 32)
```

역방향의 "마지막"은 **맨 처음 시점**. 거꾸로 읽었으니까

다음 달 예측, 다음 글자 생성에는 **쓰지 않는다**: 미래 정보 누수

11.9 패딩이 메모를 오염시킨다

```
짧은 = torch.randn(3, 2)   # 길이 3
긴 = torch.randn(6, 2)     # 길이 6
배치 = pad_sequence([짧은, 긴],
        batch_first=True)  # (2, 6, 2)

출력, h_n = rnn(배치)
_, h_혼자 = rnn(짧은.unsqueeze(0))
```

```
혼자 읽은 마지막 메모:
  [0.172, -0.593, 0.522, 0.458]
배치의 h_n:
  [-0.349, -0.04, 0.226, -0.086]
배치의 3번째 시점:
  [0.172, -0.593, 0.522, 0.458]
```

진짜 3글자를 읽은 뒤 **채운 0을 세 번 더** 읽으며 메모를 계속 고쳐 썼다. 회의가 끝났는데 녹음기가 계속 돌아간 셈

11.9 길이를 알려 주고 꾸러미로 싸기

```
문장들 = [torch.randn(3, 2), torch.randn(6, 2), torch.randn(4, 2)]
길이 = torch.tensor([len(s) for s in 문장들])
배치 = pad_sequence(문장들, batch_first=True)
꾸러미 = pack_padded_sequence(배치, 길이, batch_first=True, enforce_sorted=False)
출력_꾸러미, h_n = rnn(꾸러미)
출력, 출력_길이 = pad_packed_sequence(출력_꾸러미, batch_first=True)
```

```
실제 계산할 칸: (13, 2) 시점별 문장 수: [3, 3, 3, 2, 1, 1]
문장 0 (길이 3): 혼자 읽은 것과 같은가? True
문장 1 (길이 6): 혼자 읽은 것과 같은가? True
문장 2 (길이 4): 혼자 읽은 것과 같은가? True
되돌린 출력: (3, 6, 4) [3, 6, 4]
채운 칸의 출력은 0: 0.0
```

진짜 칸 3 + 6 + 4 = 13만 계산한다. RNN은 문장마다 자기 길이까지만 읽는다

4차시 · 11.9 꾸러미로 싸기 · 예제 11-rnn-lstm/b17_pack.py (핵심 부분)

11.9 여러 길이로 학습하고 더 긴 괄호로 시험

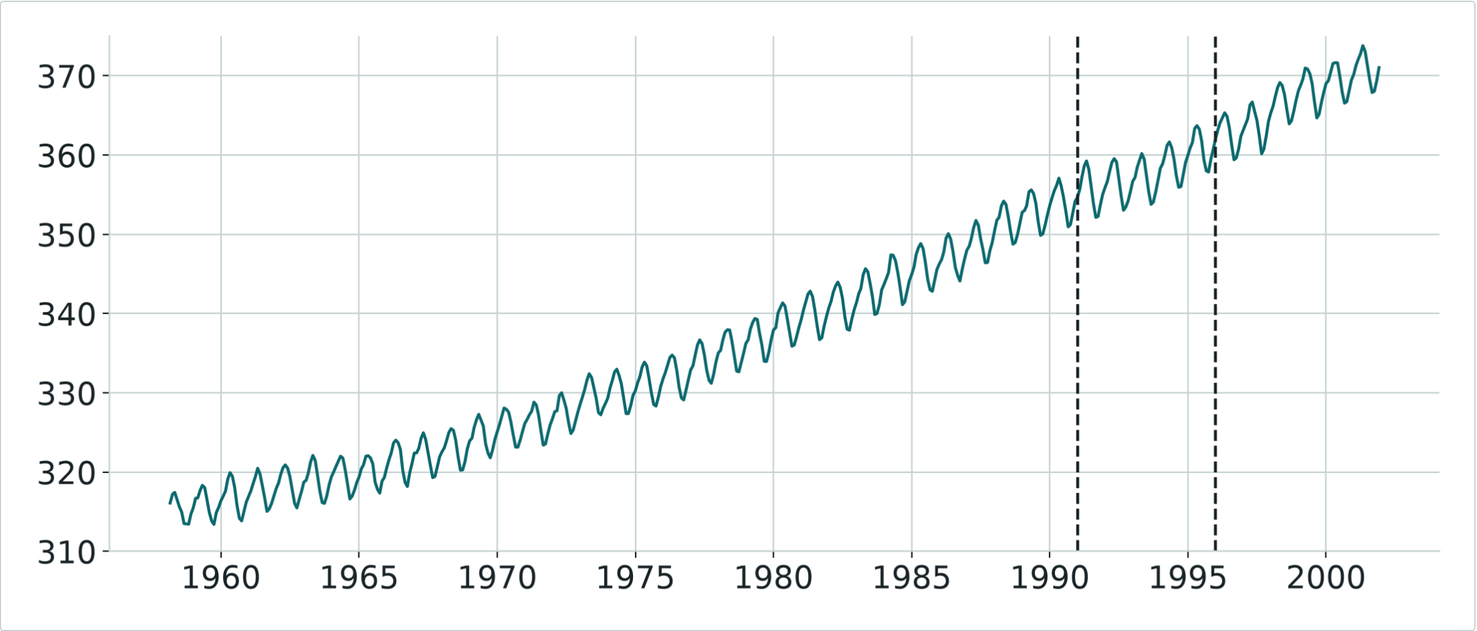
길이 2~20을 섞은 괄호 6,000개로 15에폭. 처음 보는 길이 30, 40, 60으로 시험

모델	꾸러미	검증(길이 2~20)	길이 30	길이 40	길이 60
RNN	안 씀	78.3%	62.0%	60.3%	58.1%
RNN	씀	99.9%	92.1%	79.7%	66.5%
LSTM	안 씀	100.0%	75.5%	55.0%	43.9%
LSTM	씀	100.0%	**98.2%**	**89.6%**	70.0%

RNN은 꾸러미가 없으면 학습한 길이에서도 **78.3%**

길이 30: 길이 20만 본 RNN **57.5%** → 섞고 싸면 92.1%

11.10 마우나로아 이산화탄소 둘러보기



— 월간 CO2(ppm) · 점선 = 검증 시작(1991), 테스트 시작(1996)

추세: 연평균 1960 → 2000
316.9 → 369.4

계절성: 2000년 5~6월 / 9월
371.6 / 366.5

주간 2,284개 → 월간
526개월

좋은 예측 모델은 **추세**와 **계절성**을 모두 잡아야 한다

11.10 기준선부터 세우기

```
창 = 24                                  # 지난 24개월을 보고 다음 달을 맞힌다
X, y = 값.unfold(0, 창, 1)[:-1], 값[창:]
지난달 = X[:, -1]                        # 기준선 1: 지난달 그대로
작년_같은달 = X[:, -12] + (X[:, -1] - X[:, -13])  # 기준선 2: 작년 같은 달 + 1년 증가분
```

```
학습 370 검증 60 테스트 72
지난달 그대로: 검증 RMSE 1.305, 테스트 RMSE 1.253 ppm
작년 같은 달 + 1년 증가분: 검증 RMSE 0.340, 테스트 RMSE 0.428 ppm
```

시간 순서로 자른다: **1990년까지 학습**, 1991~1995 검증, 1996~2001 테스트

사람이 만든 규칙이 테스트 **0.428ppm**. 이걸 못 이기면 신경망을 쓸 이유가 없다

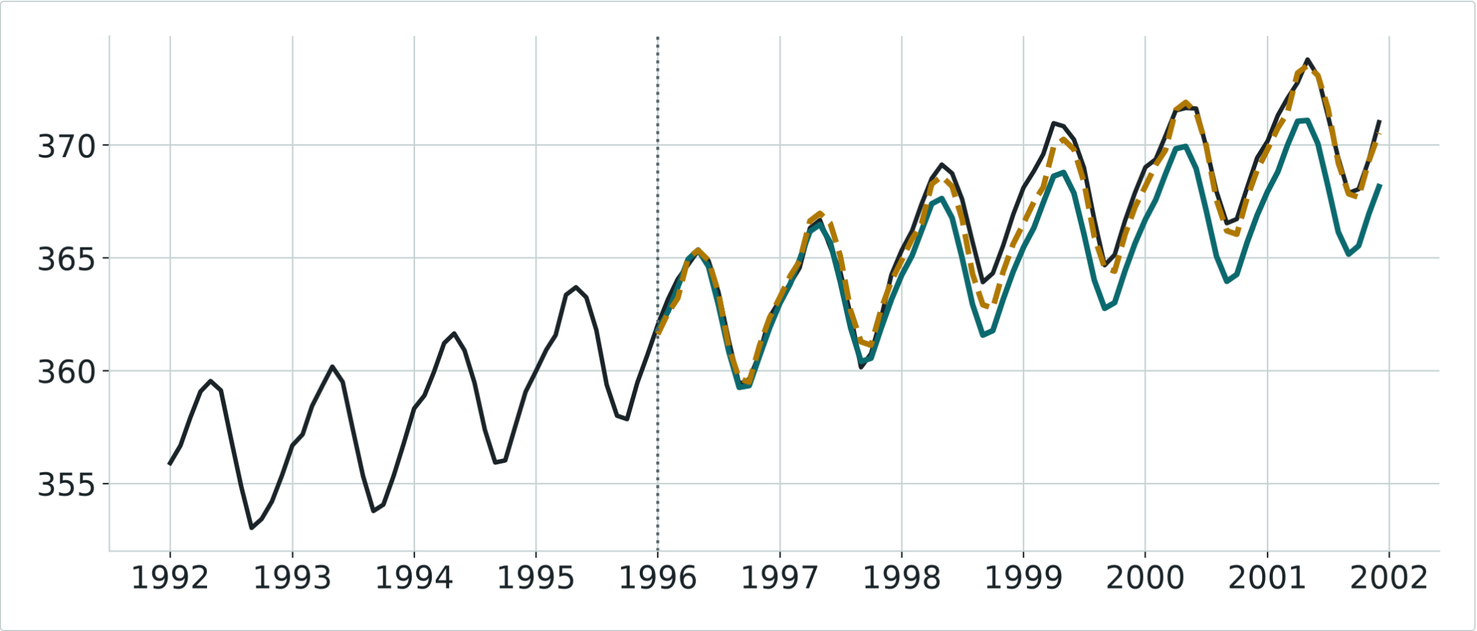
11.10 RNN, LSTM, GRU로 예측하기

```
# 절대값 대신 '마지막 달과의 차이'를 보고 '다음 달에 얼마나 변할지'를 맞힌다
마지막 = X[:, -1:]
입력, 목표 = X - 마지막, y - 마지막[:, 0]
척도 = 입력[학습].std()    # 학습 구간에서만 잰 크기로 나눈다
입력, 목표 = 입력 / 척도, 목표 / 척도
```

모델	고른 에폭	검증 RMSE	테스트 RMSE
RNN	137	**0.276**	0.325
LSTM	47	0.315	0.358
GRU	148	0.286	0.321
기준선 2	-	0.340	0.428

셋 다 기준선보다 16~25% 낮다. 검증 1등은 **평범한 RNN**: 창이 24개월이라 긴 기억이 필요 없다

11.10 여러 달을 이어서 내다보기



— 실제 · — RNN 이어 예측 · - - 기준선 이어 예측 · 점선 = 1996년 1월

첫 12개월 RMSE

RNN 0.353

기준선 0.372

72개월 RMSE

RNN 1.934

기준선 0.656

작은 오차가 **72번 쌓인다**. 한 달 뒤를 잘 맞히는 모델이 6년 뒤도 잘 맞히는 것은 아니다

직접 실행 10분

1. b21_co2_rnn.py 복사본에서 창 = 24를 12로, 저장 파일 이름을 "co2_창12.pt"로 바꿔 **먼저 실행을 걸어 둔다**. 검증 1등과 테스트 RMSE는?
2. b15_stack_bi.py 복사본의 설정들에 "세 층 양방향": dict(num_layers=3, bidirectional=True)를 더한다. h_n 모양을 먼저 예상한다
3. b17_pack.py 복사본의 문장들에 torch.randn(5, 2)를 더한다. 계산할 칸과 시점별 문장 수는?

```
cd examples/vol2/11-rnn-lstm && uv run python 내실습_창12.py
```

4차시 · 직접 실행 · 1번은 13초, 2·3번은 1초 안쪽 · 1번의 저장 이름을 꼭 바꾼다(b22가 co2_모델.pt를 읽는다)

nn.LSTM(8, 16, num_layers=2, bidirectional=True)의 h_n 첫 자리는?

Q

가 1

나 2

다 4

라 32

4차시 정리

h_n의 첫 자리는 **층 수 × 방향 수**. 양방향은 미래를 보므로 예측·생성에는 쓰지 않는다

패딩만 하면 채운 칸까지 읽어 메모가 오염된다. pack_padded_sequence(..., enforce_sorted=False)

시계열: **기준선부터**, 상대값으로 넣고, 검증으로 고른다. 이어 예측하면 오차가 쌓인다

다음 차시: 속담을 읽고 새 속담을 짓는 글자 단위 LSTM

5차시

05

실습: 속담 짓는 글자 단위 LSTM

책 11.11 ~ 11.12절

학습 목표

- 입력을 한 칸 밀어 다음 글자 맞히기 문제를 만든다
- nn.Embedding이 원-핫 @ 표와 같음을 확인한다
- 확률대로 뽑아 글을 짓고, 결과를 정직하게 평가한다

11.11 1단계: 속담 117개를 파일로

```
# 저작권이 없는 옛 속담 117개. " / "로 나눠 한 줄에 몇 개씩 적었다
속담 = """
가는 말이 고와야 오는 말이 곱다 / 가는 날이 장날
가랑비에 옷 젖는 줄 모른다 / 가재는 게 편
…
"""
속담들 = [s.strip() for 줄 in 속담.strip().splitlines() for s in 줄.split(" / ")]
경로 = Path("data/속담.txt")
경로.write_text("\n".join(속담들) + "\n", encoding="utf-8")
```

```
속담 117 개, 글자 1642 개 (줄바꿈 포함)
가장 짧은 속담: 눈엣가시
가장 긴 속담: 호랑이는 죽어서 가죽을 남기고 사람은 죽어서 이름을 남긴다
```

글자 1,642개: 아주 작은 데이터. 이 크기가 결과에 어떤 영향을 주는지 마지막에 본다

11.11 2단계: 번호로 바꾸고 한 칸 밀기

```
글자들 = sorted(set(글))  # "\n"도 한 글자
번호 = {글자: i for i, 글자
        in enumerate(글자들)}
전체 = torch.tensor([번호[글자]
        for 글자 in 글])

# 입력을 한 칸 밀면 정답이 된다
조각 = 전체[:9]
입력, 정답 = 조각[:-1], 조각[1:]
```

```
글자 종류: 346
앞쪽 다섯: ['\n', ' ', '가', '각', '간']
번호로 바꾼 글: (1642,)
  [2, 65, 1, 135, 261, 1, 19, 242]
입력: 가는 말이 고와
정답: 는 말이 고와야
```

모든 시점이 다음 글자 맞히기 문제. 정답을 따로 매길 필요 없이 글 자체가 문제와 정답이 된다. 4일차의 거대 언어 모델도 이렇게 학습한다

11.11 nn.Embedding: 번호로 줄을 꺼내는 표

```
임베딩 = nn.Embedding(len(글자들), 8)
벡터 = 임베딩(입력)
원핫 = nn.functional.one_hot(
    입력, len(글자들)).float()
torch.allclose(벡터, 원핫 @ 임베딩.weight)
```

```
임베딩 표: (346, 8)
  → 꺼낸 벡터: (8, 8)
원-핫 @ 표와 같은가? True
```

원-핫

글자 하나 = 숫자 346개, 거의 전부 0. 곱하면 한 줄만 남는다

임베딩

곱셈 없이 그 줄을 바로 꺼낸다. 표의 숫자는 학습하며 함께 고쳐진다

5차시 · 11.11 2단계: nn.Embedding · 예제 11-rnn-lstm/b24_proverb_prep.py (핵심 부분)

11.11 3단계: 임베딩 → LSTM → 다음 글자 점수

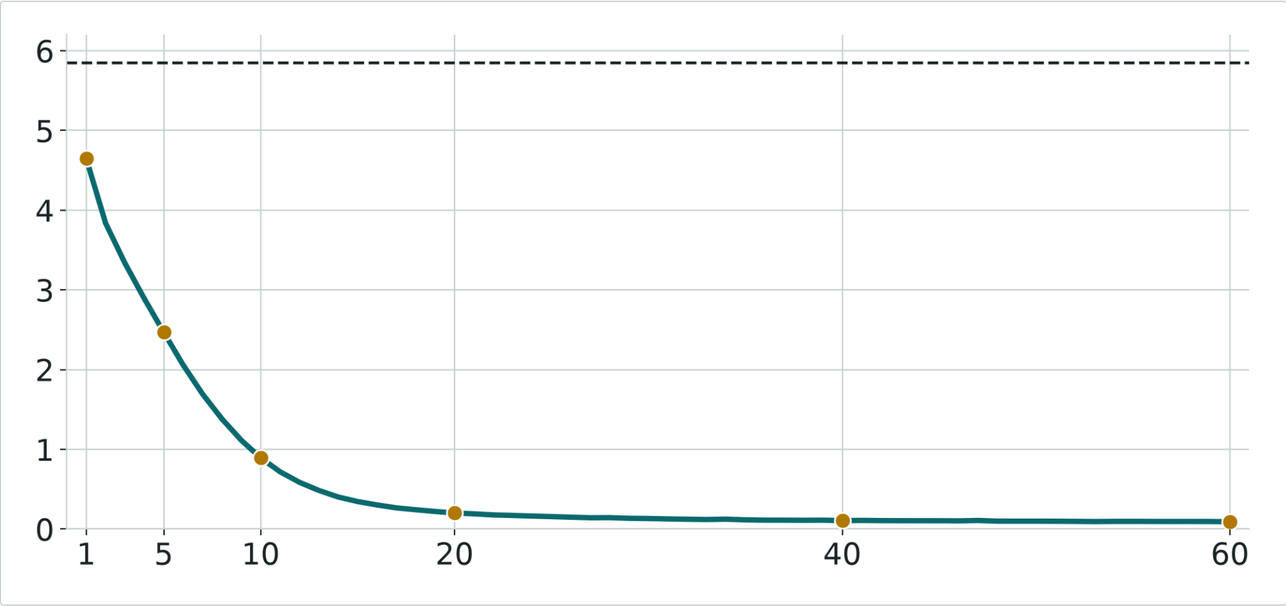
```
class 속담_LSTM(nn.Module):
    def __init__(self, 글자_수, 임베딩_크기=32, 메모_크기=128):
        self.임베딩 = nn.Embedding(글자_수, 임베딩_크기)
        self.lstm = nn.LSTM(임베딩_크기, 메모_크기, batch_first=True)
        self.다음_글자 = nn.Linear(메모_크기, 글자_수)
    def forward(self, x, 상태=None):
        출력, 상태 = self.lstm(self.임베딩(x), 상태)
        return self.다음_글자(출력), 상태  # 모든 시점에서 다음 글자 점수

조각 = 전체[묶음.unsqueeze(1) + torch.arange(길이 + 1)]  # (64, 41)
점수, _ = 모델(조각[:, :-1])
손실 = nn.functional.cross_entropy(점수.reshape(-1, len(글자들)), 조각[:, 1:].reshape(-1))
```

41글자 조각을 아무 곳에서나 잘라 앞 40은 입력, 한 칸 민 40은 정답. 펴서 **글자 하나 맞히기 2,560개**로 계산

5차시 · 11.11 3단계: 학습하기 · 예제 11-rnn-lstm/b25_proverb_train.py (핵심 부분, 27초)

11.11 손실이 줄면서 속담다워진다



— 에폭 평균 손실 · ● 글을 지어 본 에폭 · - - ln 346 ≈ 5.85

```
 1에폭 4.649 | 가을둡싼기 만 굳다글
 5에폭 2.467 | 가을 배기 있다
10에폭 0.893 | 가면 배가 있다
20에폭 0.201 | 가면 배가 있다
40에폭 0.105 | 가는 데 실 간다
60에폭 0.091 | 가는 길 먹기

"호랑이"로 시작, 40에폭:
호랑이는 죽어서 가죽을 남기고
사람은 죽어서 이름을 남긴다
```

아무 글자나 늘어놓기 → 띄어쓰기와 "~다" 말투 → **진짜 속담**

5차시 · 11.11 3단계: 학습하기 · 책의 b25_proverb_train.py와 같은 설정으로 에폭마다 손실을 기록해 그린 그림 · 오른쪽은 b25 출력

11.11 4단계: 새 속담 지어 보기

```
점수, 상태 = 모델(x, 상태)                    # 상태를 넘겨 이어서 읽는다
다음 = torch.multinomial(점수[0, -1].softmax(dim=0), 1, generator=g).item()
지은_것 = [짓기("\n", g) for _ in range(30)]  # 줄바꿈 다음 = 속담의 첫 글자부터
```

```
30개 중 원래 있던 속담 24개, 새로 지은 것 6개
 - 잘되면 제 탓 못되면 조상 리 간다
 - 호랑이도 제짝 말이 천 리 간다
 - 구는 도끼에 발등 찍힌다
 - 금은 사람은 생각도 않는데 김칫국부터 마신다
 - 땅 줄 고추가 더 맵다
 - 기는 등 건져 없는 놈 건져 놓으니 보따리 내놓으라 한다
```

두 속담에 **같은 글자 조각**("제", "말", "천 리")이 나오는 자리에서 다른 속담으로 갈아탄다

11.11 정직하게 평가하기, 그리고 한계

이해해서 짓는 것이 아니라 **외운 것을 이어 붙이는** 수준. 1,642글자, 손실 0.091은 거의 외웠다는 뜻

한계 1: 메모 하나

문장 전체를 크기가 정해진 메모 하나에 눌러 담는다. 길수록 앞부분이 흐려진다

한계 2: 한 시점씩

100번째 글자를 읽으려면 99번째까지 다 읽어야 한다. GPU의 "한꺼번에 계산"을 못 쓴다

답: 모든 자리를 한꺼번에 돌아보는 **어텐션**과 트랜스포머 (2일차)

5차시 · 11.11 결과 평가 · 11.12 다음 장 미리보기 · 책 본문 정리

직접 실행 10분

1. b23 → b25 → b26 순서로 실행한다(b25는 30초 가까이). 30개 중 원래 속담과 새것은 몇 개인가?
2. b26 복사본에서 torch.multinomial(...) 줄을 점수[0, -1].argmax().item()으로 바꾼다. 30개는 어떻게 되나?
3. b26 복사본의 시작 글자 목록을 ["바늘", "말", "개"]로 바꾼다. 어느 속담끼리 이어 붙었나?

```
cd examples/vol2/11-rnn-lstm && uv run python b25_proverb_train.py
```

글자 단위 모델의 학습 정답은 어떻게 만들었나?

Q

가 사람이 속담마다 좋다·나쁘다를 매겼다

나 입력을 한 칸 밀었다

다 원-핫 벡터를 정답으로 썼다

라 마지막 글자 하나만 정답으로 썼다

5차시 정리

입력을 **한 칸 밀면** 정답이 된다. 모든 시점이 다음 글자 맞히기 문제

nn.Embedding은 원-핫 @ 표와 같은 값을 곱셈 없이 꺼낸다

확률대로 뽑아 이어 붙인다. 작은 데이터에서는 외운 것을 이어 붙이는 수준에 머문다

다음: 1일차 마무리

5차시 · 정리 · 복습: 책 11.11 ~ 11.12절

1일차에 배운 것

1. **(배치, 길이, 특성)**, 같은 가중치로 메모를 고쳐 쓰는 RNN
2. 순서가 필요한 문제와 **기울기 소실·폭주**
3. 덧셈으로 고쳐 쓰는 **LSTM의 벨트**, GRU, 긴 기억 시험
4. 층·방향, **꾸러미**, 기준선부터 세우는 시계열 예측
5. **다음 글자 맞히기**로 학습하고 확률대로 뽑아 짓기

내일 2일차 (6~10차시)

- 어텐션, d의 제곱근
- 셀프 어텐션과 마스크
- 멀티헤드, 위치 정보, 블록 조립
- 작은 GPT로 글 만들기
- 부품 빼 보기 실습